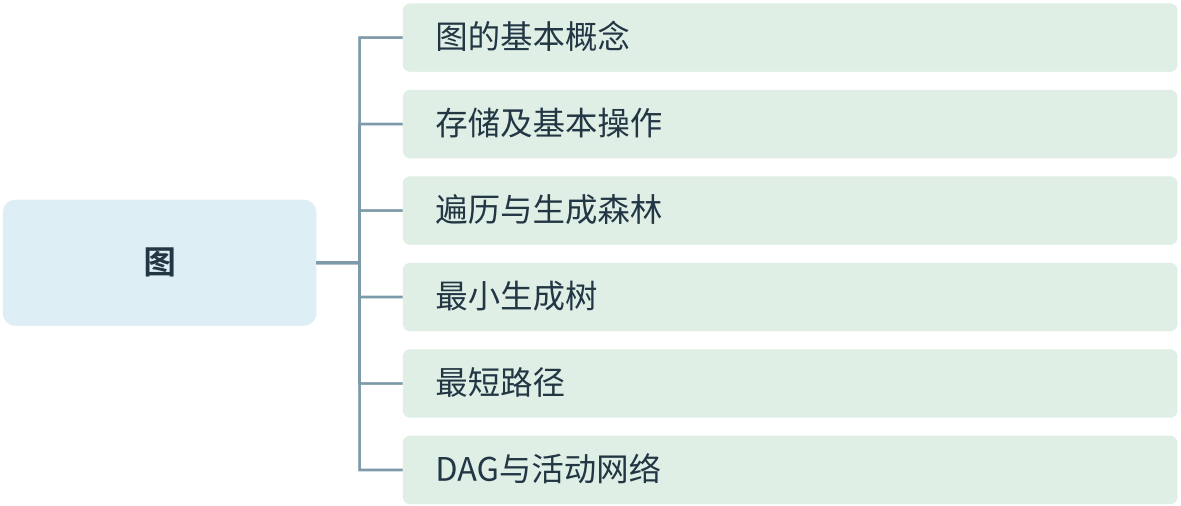
6 图

章节脉络



图的题目先辨认边的含义：一条边可以表示可通行的道路、需要铺设的线路，也可以表示任务之间的先后约束。最短路径、最小生成树、拓扑排序和关键路径分别优化或检查不同对象；选择算法前先把题目要求写成图上的条件。

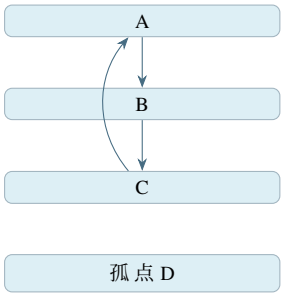
6.1 本章知识结构

- **图的基本概念**：有向无向与带权；度入度出度；路径回路与距离；连通和强连通分量；生成树森林与完全图。
- **存储及基本操作**：邻接矩阵；邻接表及逆邻接表；十字链表；邻接多重表；增删顶点与关联边。
- **遍历与生成森林**：DFS 的回退条件；BFS 的发现与队列；访问标记设置时机；遍历时间与辅助空间。
- **最小生成树**：总边权目标与适用图；Prim 的跨割状态；Kruskal 的分量合并；同权边与唯一性。
- **最短路径**：BFS 同权最短路；Dijkstra 非负权与松弛；Floyd 中转阶段；负边负环及不可达。
- **DAG 与活动网络**：共享子表达式；拓扑排序及唯一性；AOV 和 AOE 语义；事件最早最迟时间；活动余量与多条关键路径。

先明确图的方向、边权和存储表示，再跟踪访问标志、候选集合和路径信息。相同的图可以承载不同任务，判断算法时应始终回到它维护的状态与目标。

最小反例·n 减 1 条边不够判树

三点成环再加一个孤立点，共 4 顶点 3 边，恰有 $n-1$ 条边却不连通且有环。判树还要加连通或无环条件，不能只由数量推出结构。



4 顶点 3 边：三角形加孤点，满足 m=n−1 却不是树。

6.2 顶点、边、连通性与路径

课程连接·网络拓扑只是图的一种模型

计算机网络可把路由器视为顶点、链路视为边，但边权可能表示时延、费用或带宽的变换。先明确权的含义再选择算法；同一个物理网络可以对应不同优化问题。

定义 6.2.1 (图与本章模型)

图记为 $G=(V,E)$，V 为顶点集合，E 为边集合。无向边用无序顶点对表示，有向边（弧）用有序对表示。除另行说明，本章讨论无自环、无平行边的简单图，顶点数 $n=|V|>0$。无向简单图最多有 $n(n-1)/2$ 条边；有向简单图最多有 $n(n-1)$ 条弧。带权图的边还关联权值，权可以表示距离、费用或时间。

无向图顶点的度为关联边数，所有顶点度数和为 $2|E|$；有向图分别计入度、出度，各自总和都为 $|E|$。无向图度数为奇数的顶点个数必为偶数。路径长度若未加权，通常计边条数；带权长度计沿途权值和。简单路径不重复顶点，简单回路除首尾相同外不重复其他顶点。

无向图任意两顶点都有路径时称为连通图，其极大连通子图为连通分量；有向图中每对顶点相互可达时为强连通，极大强连通子图为强连通分量。单向可达不等于强连通。“极大” 指不能再加入原图其他顶点而保持相应性质，不是要求分量含全图最多的顶点。

连通无向图的**生成树**含全部顶点，连通且无环，因此有 $n-1$ 条边。它是极小连通子图：再删一边即不连通。生成树一般不是原图的连通分量，因为连通分量要保留该连通块原有的全部边，不能删到只剩一棵树。非连通图的每个分量各取一棵生成树，得到生成森林。

6.2.1 例 6-1 “有可能连通” 与 “保证连通” 不同

例 6.2.1 (配套案例)

【配套例题】无向图有 7 个顶点，要保证图在任何情况下都连通，需要的边数最少是（ ）。

A. 6　B. 15　C. 16　D. 21

解析。6 条边可以组成一棵生成树，但也可以都集中在部分顶点上，不能保证连通。若图不连通，边数最多的分法是 6 个顶点组成完全图，剩 1 个孤立顶点，共 $\binom{6}{2}=15$ 条边。超过 15 条就不可能仍不连通，故最少 16 条，答案为 **C**。

一般的 n 顶点简单无向图，边数大于 $(n-1)(n-2)/2$ 可保证连通；边数至少 $n-1$ 只是连通的必要条件。若图有 c 个连通分量，则每个分量至少要 “顶点数减 1” 条边，所以 $|E|\ge n-c$。若 $|E|<n-1$，必不连通；若 $|E|=n-1$，还需检查是否无环或连通，才能认定为树。

一张 n 顶点、n 条边的无向图必有环，因为无环森林最多有 $n-1$ 条边。但图有环并不保证连通，也不保证只有一个环。求给定边数下最多的连通分量数时，应让边尽量集中：例如 51 个顶点、21 条边，可在 7 个顶点上组成完全图，其余 44 个孤立，共 45 个分量；少于 7 个顶点最多 15 条边，无法容纳 21 条。

“完全图” 是边达到上限的图；“连通” 只要求路径存在。稠密与稀疏描述边数相对规模，具体选结构还要结合访问方式。对于 $n>1$ 的强连通有向图，每个顶点入度、出度至少 1，所以至少 n 条弧，有向环可达到下界；$n=1$ 时零弧图也可按零长度路径视为强连通，不能忽略这个边界。

有向树常指恰有一个入度为 0 的根、其余顶点入度为 1、根沿弧可达全部顶点的有向无环图。仅仅有 $n-1$ 条弧还不够。图中两点的距离是最短路径长；不可达时记为无穷，不把 “没有路径” 误记为距离 0。

6.3 邻接矩阵、邻接表与基本操作

错项排除·零权边不是没有边

带权邻接矩阵用无穷表示无边时，零是合法边权。若仍以 if(A[u][v]) 判断有边，就会漏掉零权边；AOV 的 0/1 邻接矩阵与这种表示的语义不同。

邻接矩阵 `A[u][v]` 表示是否存在 $u \to v$；带权时元素存放权值，用 $+\infty$ 表示不存在，不能把 0 既用作 “无边” 的标记，又用作零权边的权值。无向图矩阵对称，有向图不必。矩阵占 $\Theta(n^2)$ 空间，检查一条给定边为 $O(1)$，枚举某顶点邻边通常要扫整行 $O(n)$。

邻接表为每个顶点保存其邻接点链。无向图每条边通常出现两个表结点，有向图每条弧在出边表出现一次，空间均为 $\Theta(n+|E|)$。枚举邻边只访问实际表项，判断一条边是否存在则可能扫描相应链。逆邻接表以终点组织入边，方向与普通出边表相反。

下图同时用于说明存储表示与遍历。无向边为 `ab`、`ac`、`ae`、`be`、`cf`、`df`、`de`，各邻接表按字母升序排列。

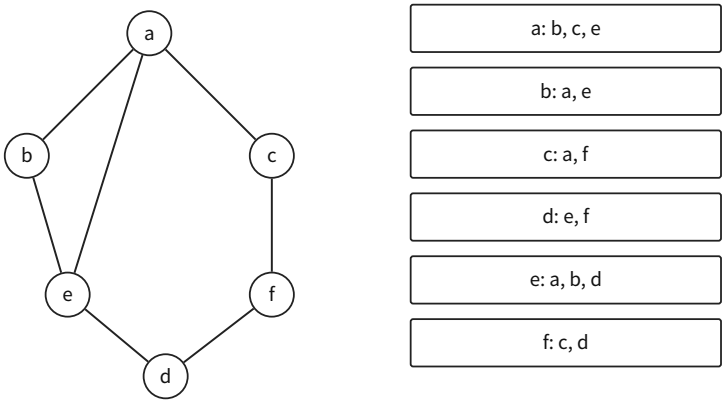


Figure 6.1: 本章遍历图；实线表示无向边，左侧表保存每个顶点的邻接点

对该图，a 行有 b、c、e 三个非零项，b 行有 a、e 两个。无向图中，顶点的度等于该行非零项的个数；有向图中，行的非零项数给出出度，列的非零项数给出入度。求有向图所有顶点的入度时，矩阵要扫描 n^2 个位置，邻接表只需扫描所有表项，成本不同。

删除顶点会同时删除其所有关联边。无向邻接表不能只删除该顶点自己的链，还要清除其他顶点中指向它的表项；有向图则须同时考虑入边和出边。题目若给出 “只按出度计删除成本” 的实现，应检查它是否另有入边索引。

6.3.1 顶点表、边表与基本操作接口

下面保留教材的矩阵和邻接表命名。`VertexType` 为顶点信息类型，`EdgeType` 为边权类型；实际使用前要根据问题定义它们。算法中的顶点编号是数组下标 `0..vexnum-1`，图上显示的 V_1, V_2 等名称可以另存于顶点信息中。

```c
#define MaxVertexNum 100
typedef struct {
    VertexType Vex[MaxVertexNum];
    EdgeType Edge[MaxVertexNum][MaxVertexNum];
    int vexnum, arcnum;
} MGraph;

typedef struct ArcNode {
    int adjvex;
    struct ArcNode *next;
    // 带权图在此补充权值域
} ArcNode;
typedef struct VNode {
    VertexType data;
    ArcNode *first;
} VNode, AdjList[MaxVertexNum];
typedef struct {
    AdjList vertices;
    int vexnum, arcnum;
} ALGraph;
```

条件辨析·编号与顶点信息

算法使用的 0 起始编号是数组下标。图上 V1 可以对应下标 0，也可以另存信息；转换一次后整章保持一致。

`arcnum` 记录图中的实际边或弧条数；无向邻接表即使有 $2|E|$ 个边表结点，也仍记录 $|E|$，不能按表结点总数填入。矩阵的 0/1 表示与带权表示应分别初始化：带权对角线在最短路径初始化中通常取 0，无边的非对角位置取无穷。

教材用下列接口屏蔽具体存储：

- `Adjacent(G,x,y)`判断边存在，`Neighbors(G,x)`枚举邻边。
- `InsertVertex`、`DeleteVertex` 增删顶点；`AddEdge`、`RemoveEdge` 增删边。
- `Get_edge_value`、`Set_edge_value` 读写指定边的权值。
- `FirstNeighbor(G,x)`求第一个邻点；`NextNeighbor(G,x,y)`求邻点 y 之后的下一个邻点，无对应邻点时返回 -1。

以下对这些接口采用同一方向约定：有向图的邻点指当前顶点的出弧所到达的顶点。

操作	邻接矩阵	普通出边邻接表
判断 $x \to y$ 是否存在	$O(1)$	扫描 x 的出边链
枚举 x 的全部出邻点	$O(\|V\|)$	$O(OD(x))$
求某点出度	扫一行，$O(\|V\|)$	数该链结点，$O(OD(x))$
求某点入度	扫一列，$O(\|V\|)$	扫全体出边表，$O(\|V\|+\|E\|)$
删除顶点	删除关联行列并处理编号	清除出链与所有指向它的入弧

有向图若另建逆邻接表或十字链表，求入度和删除入弧的代价可随之改变。无向邻接表查度数直接数对应链即可。

无向图矩阵对称，可只保存上三角或下三角，再按对称关系恢复另半。若连同对角线保存，需要 $n(n+1)/2$ 个单元；简单图的对角线已知全为 0、另作约定时，可只存严格三角的 $n(n-1)/2$ 项。压缩节省的是常数比例，空间阶仍为 $O(n^2)$；定位公式须与保存方向和 0 ／ 1 起始下标一致。

6.3.2 矩阵幂数的是游走

设 A 是非加权图的 0/1 邻接矩阵，按普通整数矩阵乘法计算，则 $(A^k)_{ij}$ 等于从 i 到 j、恰好经过 k 条边的**游走数**，允许重复顶点和边。这来自乘法的逐中间点求和：$(A^2)_{ij}=\sum_t A_{it}A_{tj}$，每个非零乘积对应一条 $i\to t\to j$。

例如只有无向边 $1-2$ 时，$(A^2)_{11}=1$，对应 $1\to 2\to 1$；它不是从 1 回到 1 的简单路径。题目若把这里的“路径”用作可重复的广义路径，应按这一含义理解。布尔矩阵乘法得到是否存在，普通算术乘法得到条数，二者也应区别。

不变量·一条边结点参与两条链

邻接多重表让同一个无向边结点同时属于两个端点链。沿当前端点前进时选择其对应链接；这解释了为何可统一标记边，也解释了不能始终沿 ilink。

6.4 十字链表与邻接多重表

十字链表面向有向图，每条弧只建立一个弧结点，记录弧尾、弧头以及两个链接：一个沿相同弧尾串联出弧，一个沿相同弧头串联入弧。每个顶点分别保存第一条出弧、第一条入弧。这样同一条弧可从源顶点的出边链或终点的入边链到达，无需存两份弧数据。

邻接多重表面向无向图，每条边只建一个边结点，记录两个端点 `ivex,jvex`，并有 `ilink,jlink`，分别串入两个端点的关联边链。沿顶点 u 的链前进时，须先判断 u 是这条边的哪个端点，再选择对应链接；不能始终只走 `ilink`。这种表示便于给一条边设置统一访问标记、执行删除等边操作。

普通邻接表为每条无向边保存两个表项；邻接多重表中的一个边结点同时挂在两条链上。数边时若直接数链表项，两种结构得到的数目会相差常数倍。稀疏矩阵的十字链表也使用“两种方向共享一个数据结点”的思想，但它的两维是行与列，不是有向图的弧尾与弧头。

6.5 深度优先搜索与搜索森林

DFS 从一个顶点开始，标记并访问，再递归访问尚未访问的邻接点；当前顶点已没有未访问邻点时才回退。标记用于防止沿环重复访问。遍历整张图时，还须依次检查所有顶点，对仍未访问的顶点重新启动搜索；一趟 DFS 未必到达全图。

```
bool visited[MaxVertexNum];
void DFS(Graph G, int v) {
    visit(v);
    visited[v]=true;
    for (int w=FirstNeighbor(G,v); w>=0;
         w=NextNeighbor(G,v,w))
        if (!visited[w]) DFS(G,w);
}
void DFSTraverse(Graph G) {
    for (int v=0; v<G.vexnum; ++v) visited[v]=false;
    for (int v=0; v<G.vexnum; ++v)
        if (!visited[v]) DFS(G,v);
}
```

上图按字母顺序枚举邻点，从 a 开始，得到 `a,b,e,d,f,c`。首次到达新顶点的边形成 DFS 生成树；无向图的搜索树棵数等于连通分量数，有向图不能用树棵数直接认定强连通分量数。

6.5.1 例 6-2　什么时候允许回退去另一个分支

例 6.5.1 (配套案例)

〖6.3-011〗无向图顶点为 `{a,b,c,d,e,f}`，边为 `{ab,ae,ac,be,cf,fd,ed}`。进行深度优先遍历，不能得到的序列为（　）。

A. acfdeb　B. aebdfc　C. aedfcb　D. abecdf

解析。D 访问到 `a,b,e` 时，e 的邻点 d 尚未访问，必须深入 d，不能先回退到 a 转去 c，因此 D 不可能。A 可沿 a→c→f→d→e→b；B 可先走 a→e→b，b 无未访问邻点后回到 e，再走 d→f→c；C 可先走 a→e→d→f→c，回退后再处理 b。答案为 **D**。输出序列中相邻的两个顶点不一定直接相连，因为中间可能发生回退；判断时真正要看的是，回退发生时当前顶点是否还有未探索的邻点。

采用邻接表时，每个顶点和表项只处理常数次，时间 $O(n+|E|)$；矩阵要对每个访问顶点扫一整行，时间 $O(n^2)$。递归栈最坏 $O(n)$，另有访问标志数组 $O(n)$。如果改为在递归返回前才输出顶点，得到的是完成次序；对 DAG，该完成次序的逆序就是拓扑序列，第 8 节会说明原因。

6.5.2 遍历序列、存储次序与生成森林

`Graph` 在遍历代码中表示已经实现上述邻点接口的图类型；选用矩阵或邻接表后，由对应接口完成枚举。只有**起始点、外层顶点扫描顺序、邻接点枚举顺序**都固定，遍历序列才固定。对编号固定的邻接矩阵，常规逐列递增扫描会给出一种确定顺序；若改变扫描方向，同一矩阵也可得到另一序列。

每个新顶点第一次被发现时，记录其前驱，便得到搜索生成树。无向图中一棵树覆盖一个连通分量，若有 c 个分量，森林共 $|V|-c$ 条树边。非树边仍属于原图，不能因它未出现在搜索树中就从图里删除。有向图的搜索树覆盖的是这次沿出弧可达且尚未访问的部分，外层启动次数受起点次序影响，不能拿来计算强连通分量。

DFS 的递归深度、BFS 的队列长度都可能达到 $O(|V|)$；把访问标志数组算入辅助空间时，完整遍历至少还需 $O(|V|)$ 标志空间。即便某个 DFS 实例的调用栈只有常数深度，也不能把整个实现的辅助空间一并写为 $O(1)$。

6.6 BFS 为什么得到无权最短路

BFS 用队列保存已发现、尚未扩展的顶点。起点先标记并入队；每次取出队头，将其尚未发现的邻点标记、记录距离并入队。标记应在**入队时**设置，若等到出队才标记，同一顶点可能被不同前驱重复入队。

最小反例·边最少而权更大

三点 A、B、C，AB 与 BC 权 1，AC 权 3。BFS 从 A 一跳发现 C，少一条边却总权 3；经 B 两跳总权 2。因此 BFS 的分层证明不能覆盖不等权路径目标。

```
void BFS(Graph G, int v) {
    SqQueue Q;
    InitQueue(Q);
    visit(v);
    visited[v]=true;
    EnQueue(Q,v);
    while (!QueueEmpty(Q)) {
        DeQueue(Q,v);
        for (int w=FirstNeighbor(G,v); w>=0;
             w=NextNeighbor(G,v,w)) {
            if (!visited[w]) {
                visit(w);
                visited[w]=true;
                EnQueue(Q,w);
            }
        }
    }
}
void BFSTraverse(Graph G) {
    for (int v=0; v<G.vexnum; ++v) visited[v]=false;
    for (int v=0; v<G.vexnum; ++v)
        if (!visited[v]) BFS(G,v);
}
```

条件辨析·三个顺序共同确定遍历

起始顶点、外层扫描次序、邻点枚举次序都固定，DFS 或 BFS 才有固定输出。矩阵存储本身没有禁止逆序扫描。

上面的队列元素为顶点编号，容量要能容纳全部可能同时入队的顶点。若采用第 3 章牺牲一个单元的循环队列，物理数组须比允许队列长度多 1。

求从源点 u 出发的无权最短路径时,初始化全部 `d[i]=INF`、`path[i]=-1`，令 `d[u]=0`。在发现边 $v \to w$ 且 `visited[w]==false` 的分支中增加:

```
d[w]=d[v]+1;
path[w]=v;
```

每个顶点只在首次发现时赋距离。`path[w]`是其路径前驱，可从目标反向追到源点；最后把这条逆序链倒过来。源点 `path[u]=-1` 是终止标志，不表示源点不可达。

BFS 首次发现即最短

队列保证按距离分层出队：距离 0 的顶点先于距离 1 的，距离 1 的又先于距离 2 的。首次发现 v 时，若还存在更短的路径，这条路径上 v 的前驱应位于更早的一层，早已被扩展，v 也就应更早被发现，矛盾。因此 `dist` 记录的是最少边数。

□

上图从 a 出发、邻点按字母枚举，出队访问序列为 `a,b,c,e,f,d`；b、c、e 距离 1，f、d 距离 2。该序列与 DFS 顺序不同，但也不唯一：同层邻接次序可改变，最少边数不变。带权不等长边时，少经过一条边未必总权更小，不能把这条证明直接推广为带权最短路。

邻接表与矩阵的时间同 DFS，队列和标记等辅助空间为 $O(n)$。有向图中，BFS 只沿弧的方向扩展；“无权最短路”并不要求图无向，但要求每条边的代价相同，只按边数计。

6.6.1 王道课件的八顶点最短路径实例

下图中的顶点名称为 1～8，以 2 为源点，邻点按名称递增枚举。源点先发现 1 和 6；随后 1 发现 5，6 发现 3 和 7；再由 3 发现 4、由 7 发现 8。BFS 生成树保留每个非源点的首次发现前驱，其根到各点的路径具有最少边数。

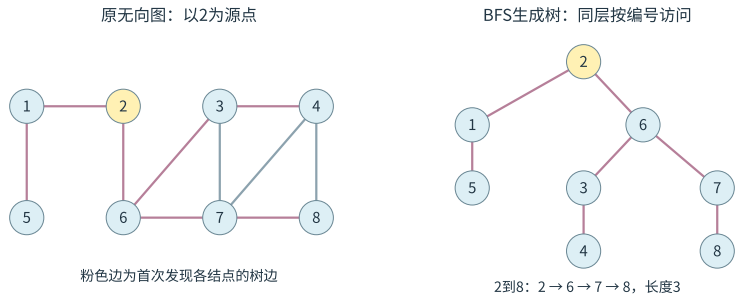


Figure 6.2: 八顶点原图与以 2 为根的广度优先生成树

顶点名称	1	2	3	4	5	6	7	8
距离 d	1	0	2	3	2	1	2	3
前驱 path	2	−1	6	3	1	2	6	7

从目标 8 倒推前驱，得到 $8 \leftarrow 7 \leftarrow 6 \leftarrow 2$，反转即为最短路径 $2o6o7o8$，长度 3。表中使用图上的 1 起始名称，写入前述 0 起始图接口时，应把名称映射为数组下标；源点的前驱 −1 保留为终止标记。

把题意翻译成图算法的目标

先确定有向还是无向、边权是否同一常数，再问目标是“全部顶点以最低总价接通”“某个起点到各点最短”“先后约束合法”还是“工程完成时间”。BFS 只保证同权边最短，MST 只控制整棵树总权；它们不能互相替代。

6.7 Prim 与 Kruskal 在挑什么边

最小生成树（MST）要求连通全部顶点、无环，并使所选 $n-1$ 条边权和最小。它不要求从某个起点到每个顶点的路径最短。若图不连通，只能得到每个分量的最小生成森林。

Prim 维护一个已连接顶点集合 U，每次从“一端在 U、一端在外”的边中选最小边，加入新顶点。Kruskal 则把全部边按权非降序考虑，端点在不同当前分量时选入并合并，同一分量则跳过以避免成环。二者都利用割上的轻边性质，但状态不同：Prim 始终扩展一棵树，Kruskal 可能同时有多棵。

6.7.1 例 6-3　同权边怎样影响第二次选边

例 6.7.1 (配套案例)

【配套例题】求下图最小生成树，可能为 Kruskal 第 2 次选中、但不可能为从 V_4 开始的 Prim 第 2 次选中的边是（　）。

错项排除·同权不等于 MST 必不唯一

若所有边权互异，MST 一定唯一；逆命题不成立。一个本来就是树的连通图，即使边权相同，其生成树仍只有它本身，不能根据同权直接判不唯一。

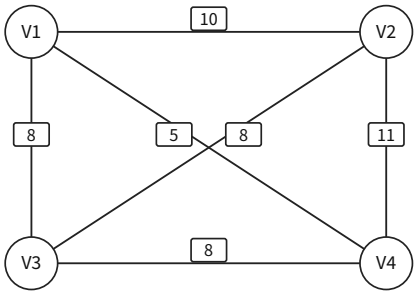


Figure 6.3: 四顶点带权无向图；三条权为 8 的边形成选择分支

A. (V_1, V_3)　B. (V_1, V_4)　C. (V_2, V_3)　D. (V_3, V_4)

解析。两个算法首先都选权 5 的 (V_1, V_4)。Kruskal 此后可选任意一条不成环的权 8 边，包括 (V_2, V_3)。Prim 此时 U 为 $\{V_1, V_4\}$，第 2 条边必须跨越这个集合的边界，只能在 (V_1, V_3) 与 (V_4, V_3) 中选权 8 的边；(V_2, V_3) 两端都在集合外，不能选。答案为 **C**。

继续执行时，Kruskal 若选 (V_2, V_3)，最后还要用一条权 8 边把两个分量连接，总权 21；Prim 选入 V3 后，再用 (V_3, V_2) 权 8 连 V2，也是 21。同权边可能导致不同的 MST；全部边权互异足以保证 MST 唯一，但存在同权边时 MST 仍可能唯一。

朴素 Prim 配矩阵为 $O(n^2)$；邻接表配专门维护候选最小值的结构可降低稀疏图中选最小边的代价。Kruskal 主要花在边排序，常写 $O(|E| \log |E|)$，加并查集维护分量。具体选哪一个，还要看图的表示与选边的实现，不能只比较算法名字。

6.7.2 最小生成树的核心伪码

```
Prim(s):
    U={s}; lowCost[v]=w(s,v)，无边记无穷
    重复直到选入全部顶点:
        u=U外lowCost最小者
        若lowCost[u]=无穷: 原图不连通，停止
        选入parent[u]到u的边; U加入u
        对每个U外邻点v:
            若w(u,v)<lowCost[v]: 更新lowCost[v]及parent[v]
```

这里 `lowCost[v]`始终是 U 到 v 的最轻跨割边，**不是**从 s 到 v 的路径长度，所以更新式没有 `lowCost[u]+w(u,v)`。

```
Kruskal(G):
    将边按权非降排序；每个顶点初始化为单独集合
    对每条边(u,v,w):
        若find(u)!=find(v):
            选入此边; unite(u,v)
    若选入边数<n-1: 得到最小生成森林而非生成树
```

为何跨割最轻边可以安全加入

不变量·Prim 的 key 不是距离

Prim 只问一条跨割边接新点的代价，key 更新取边权；Dijkstra 问源点到新点的完整路径，需加已确定前缀。把两者公式混写可能算出另一种对象。

设已有选择能扩展到某棵 MST，考虑与现有分量相容的割及其一条轻边 e。若某 MST 不含 e，加入 e 形成环，环上必有另一条跨该割的边 f，且 $w(e) \le w(f)$。删 f 保连通且不增权，得到含 e 的 MST；这说明选择可扩展，不表示所有 MST 都必须含 e。

□

错项排除·有负边与一定算错不同

负边意味着标准 Dijkstra 失去通用正确性保证，不代表每个含负边实例都会得错值。选项若用“只要有一条负边就必算错”的绝对语气，可用简单单路径反驳。

6.8 Dijkstra 与 Floyd 的条件和中间状态

Dijkstra 求单源最短路径。维护已确定集合 S 与暂定距离 `dist[v]`；每次选择 S 外距离最小的顶点 u，确定它的最短距离，再用 `dist[u]+w(u,v)` 松弛其邻点。尚未确定的 `dist` 是已知可达路径的最好上界，不一定已是真正最短距离。

通用正确性要求边权非负：若能先经过一个当前距离更远的顶点、再经负边绕回 u，u 的真实距离就可能小于已确定的值，于是 u 被过早确定。出现负边不表示每个实例都必然算错，但标准算法不能保证正确。不可达顶点的距离保持无穷，所有剩余距离都为无穷时可停止，不能再把无穷当作普通数参与加法。

6.8.1 例 6-4　求出第二条最短路径后哪些值还可变

例 6.8.1 (配套案例)

【配套例题】使用 Dijkstra 求下图从顶点 1 到其余顶点的最短路径，`dist` 依次保存到 2、3、4、5 的距离。求出第二条最短路径后，`dist` 内容为（　）。

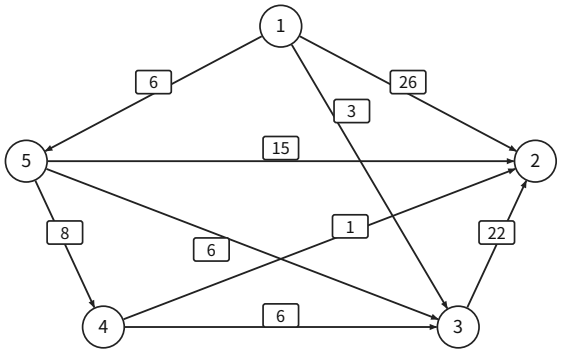


Figure 6.4: 原题有向边与权值；箭头方向参与松弛条件

A. 26、3、14、6　B. 25、3、14、6　C. 21、3、14、6　D. 15、3、14、6

解析。先把起点 1 确定，初始 `dist=(26,3,∞,6)`。第 1 条到其他顶点的确定路径是 1→3，长 3；用 3→2 的 22 更新到 2 距离为 25。随后选顶点 5，距离 6，第 2 条路径确定；由 5→2 的 15 把到 2 更新为 21，由 5→4 的 8 把到 4 更新为 14，得到 **(21,3,14,6)，答案 C**。

已确定的新顶点	暂定距离向量 (dist[2],dist[3],dist[4],dist[5])
1（初始化）	(26,3,∞,6)
3	(25,3,∞,6)
5	(21,3,14,6)
4	(15,3,14,6)
2	(15,3,14,6)

不变量·已确定与暂定要分开

Dijkstra 中暂定 dist 只是一条已知路径的最好上界。选出最小暂定点后才永久确定；过程题若提前使用未来松弛的值，就把最终答案错误放入了中间状态。

答案要求的是第 2 条路径确定后，不是算法最终结果。顶点 4 随后才用其到 2 的权 1 把 21 改为 15，不能把这个将来更新提前放进答案。

Floyd 求任意顶点对之间的最短路。令 $D^{(k)}[i,j]$ 表示只允许编号不超过 k 的顶点作为内部中转时的最短距离，则

$$D^{(k)}[i,j]=\min\{D^{(k-1)}[i,j],D^{(k-1)}[i,k]+D^{(k-1)}[k,j]\}.$$

两种候选分别是不经过 k，或在 k 处分成两段。实现时中转顶点 k 必须放在最外层循环，才能对应这个阶段的含义。时间 $O(n^3)$，距离矩阵 $O(n^2)$；允许负边，但要求不存在影响所求路径的负权回路，否则不存在有限最短值。无向图中一条负权边就能往返构成负权闭合游走，不能直接套用通常的有限最短路模型。

矩阵版 Dijkstra 单源时间 $O(n^2)$；对每个源点重跑得到全源需 $O(n^3)$，不能仍写成 $O(n^2)$。BFS 求的是同权边下的最少边数，MST 最小化整棵树的边权总和，Floyd/Dijkstra 最小化指定端点之间的路径权和；三个目标要分别辨认。

6.8.2 最短路径的阶段循环

```
Dijkstra(s):
    dist[s]=0; 其余为无穷; S为空
    重复选择S外dist最小的u:
        若dist[u]=无穷: 停止
        S加入u
        对每条u到v，若v未确定且dist[u]+w(u,v)<dist[v]:
            更新dist[v]; parent[v]=u
```

最外层每次永久确定一个顶点，这是非负权前提的用处；已经确定的点不能靠未来负边回头改写。实现中先检查 `dist[u]`有限，避免无穷哨兵加权溢出。

```
Floyd(D):
    D[i][i]=0; 其他位置按直接边初始化
    for k=0..n-1:
        for i=0..n-1:
            for j=0..n-1:
                若D[i][k]与D[k][j]均有限:
                    D[i][j]=min(D[i][j],D[i][k]+D[k][j])
```

k 在外层时，“已允许的中间顶点集合”才依次扩大。若结束后某 `D[v][v]<0`，说明存在负环；受该负环影响的点对没有有限最短距离，不能继续解释该数值为答案。

6.8.3 Floyd 三阶段计算实例

条件辨析·path 有两种语义

Floyd 的 path[i][j] 记录中转点；BFS 或 Dijkstra 的一维 path[v] 通常记录直接前驱。重建路径时分别递归拆分或反向追溯。

课件的三顶点有向图，按 V_0, V_1, V_2 排列，直接边为 $V_0 \to V_1 = 6$、$V_0 \to V_2 = 13$、$V_1 \to V_0 = 10$、$V_1 \to V_2 = 4$、$V_2 \to V_0 = 5$。无边记 ∞，对角线为 0：

$$A^{(-1)} = \begin{pmatrix} 0 & 6 & 13 \\ 10 & 0 & 4 \\ 5 & \infty & 0 \end{pmatrix}.$$

允许 V_0 中转后，$2 \to 1$ 由 ∞ 更新为 $5 + 6 = 11$；允许 V_1 中转后，$0 \to 2$ 由 13 更新为 $6 + 4 = 10$；允许 V_2 中转后，$1 \to 0$ 由 10 更新为 $4 + 5 = 9$：

$$A^{(0)} = \begin{pmatrix} 0 & 6 & 13 \\ 10 & 0 & 4 \\ 5 & 11 & 0 \end{pmatrix},$$

$$A^{(1)} = \begin{pmatrix} 0 & 6 & 10 \\ 10 & 0 & 4 \\ 5 & 11 & 0 \end{pmatrix},$$

$$A^{(2)} = \begin{pmatrix} 0 & 6 & 10 \\ 9 & 0 & 4 \\ 5 & 11 & 0 \end{pmatrix}.$$

中转记录初值 `path[i][j]=-1`。每次因 k 而更新距离，就令 `path[i][j]=k`；例如最终 `path[0][2]=1`，说明 0 到 2 应拆为 0 到 1、1 到 2。若子段还有中转点，再递归拆分。这个矩阵保存**中间顶点**，与 Dijkstra 或 BFS 保存 “直接前驱” 的一维 `path` 含义不同。

6.8.4 DAG 怎样共享子表达式

表达式树为每次运算保留一份子树；DAG 可以让多个父运算共享相同子表达式。对 `(a*b)*(a*b)*(a*b)*c`，先为操作数 a, b, c 各建一个顶点，建立一次 $t_1 = a * b$，再按左结合次序建立 $t_2 = t_1 * t_1$、$t_3 = t_2 * t_1$、$t_4 = t_3 * c$。两个操作数相同的乘法也保留左右两个输入位置，即便它们指向同一个 t_1。

构造时先标运算依赖，再合并相同操作数和相同运算的相同输入子图；只有操作符和有序输入都相同，才可直接共享。`a-b` 和 `b-a` 不能合并；`a/(b*c)`与 `(a/b)*c` 也不能按视觉相似合并。含赋值、函数调用等副作用时，还须遵守表达式求值语义。

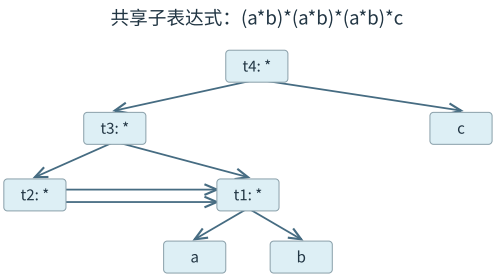


Figure 6.5: 同一个 a*b 结点分别被平方和后续乘法引用，边指向输入子表达式

6.9 拓扑排序：由入度判断哪些任务可以先做

不变量·入度为零才可先执行

入度表示尚未完成的直接前驱数；输出顶点并移除其出弧后，后继入度随之递减。队列为空而顶点尚未全部输出时，图中存在有向环。

有向无环图（DAG）可以用顶点表示活动、弧表示先后约束，称为 AOV 网。拓扑序列要求对每条弧 $u \to v$，u 都排在 v 之前。算法先把所有入度为 0 的顶点作为候选，每次删除一个候选及其出弧，更新后继入度；若处理完的顶点数不足 n，则存在有向环。

有多个候选时可以选择不同的顺序，因此拓扑序列一般不唯一。对 DAG 做 DFS 并在退出时输出，每条弧的终点必先完成，得到的是逆拓扑序列；把该完成序列反向才是拓扑序列。若存在有向环，仅取得一个 DFS 完成次序并不能证明拓扑合法。

6.9.1 例 6-5　怎样判定唯一拓扑序列

例 6.9.1 (配套案例)

【配套例题】载人航天工程包含众多子工程，为保证有序开展，需要明确前导子工程。这可抽象为有向图的拓扑序列问题。已知图 G 采用邻接矩阵存储：

```
typedef struct {
    int numVertices,numEdges;
    char VerticesList[MAXV];
    int Edge[MAXV][MAXV];
} MGraph;
```

设计 `int uniquely(MGraph G)` 判定 G 是否存在唯一拓扑序列，是则返回 1，否则返回 0。给出设计思想及带关键注释的 C 或 C++ 实现。

解析。每一轮的候选零入度顶点必须恰有一个。若图无环且某轮至少有两个，二者都可作为下一项，交换其先后可产生至少两个拓扑序列；若某轮没有候选而尚未处理完，则余图有环，无法完成。这两种情况都不满足唯一，均可返回 0。只有始终唯一且处理完全体顶点，才能返回 1。

```
int uniquely(MGraph G) {
    int n=G.numVertices, indeg[MAXV]={0};
    bool used[MAXV]={false};
    for (int u=0; u<n; ++u)
        for (int v=0; v<n; ++v)
            if (G.Edge[u][v]) ++indeg[v];
    for (int step=0; step<n; ++step) {
        int pick=-1;
        for (int v=0; v<n; ++v)
            if (!used[v] && indeg[v]==0) {
                if (pick!=-1) return 0; // 至少两个候选
                pick=v;
            }
        if (pick==-1) return 0;          // 剩余部分有环
        used[pick]=true;
        for (int v=0; v<n; ++v)
            if (G.Edge[pick][v]) --indeg[v];
    }
    return 1;
}
```

课程连接·同步前驱约束就是偏序

操作系统中多个事件的前驱关系可用 DAG 表示，零入度事件代表当前所有前驱已完成。拓扑排序只给合法先后，不计算耗时；带持续时间的工期还需 AOE 时间传播。

这里 AOV 邻接矩阵用 0/1 分别表示无边与有弧，不能直接把带权矩阵的 “无边为无穷” 搬进这个判断。总时间 $O(n^2)$，工作数组额外占 $O(n)$；题给的按值传参若实际复制整个矩阵，形参副本还占 $O(n^2)$，可按接口许可改为只读引用。若使用邻接表并维护零入度候选数，处理时间可为 $O(n+|E|)$。顶点已经按某个拓扑次序编号时，所有弧才保证由小编号指向大编号，邻接矩阵才呈严格上三角；任意编号的 DAG 不必原样为三角矩阵。

DAG 还可表达共享子表达式：相同子式只存一份，不同上层结点可指向同一子式。它表达计算依赖而非普通树的唯一双亲关系；不能再套用 “除根外每结点恰有一个双亲”。

6.9.2 一般拓扑排序的队列实现

```
topological(G):
    统计所有顶点的入度；把入度0者入队Q
    count=0
    while Q非空:
        u=出队；输出u；count++
        对每条u到v: indeg[v]--
            若indeg[v]==0: v入队
    返回count==n
```

入度表示余图中尚未删除的前驱数，只有从 1 降到 0 时加入候选。若只关心任意拓扑序列，可以任选候选；要判断唯一，则每一步候选数都必须恰为 1，原例的代码正是在增加这个约束。

6.9.3 逆拓扑排序与 DFS 环检测

从出度为 0 的顶点开始，删除顶点及其入弧，得到逆拓扑序列。普通出边邻接表查找入弧不便，可使用逆邻接表维护出度变化；若直接每轮扫描全图，复杂度会改变。DFS 在返回前输出顶点，也能在 DAG 上得到逆拓扑序列。

判断有向环时，把顶点分为未访问、正在递归栈中、已完成三种状态。沿弧到达“正在递归栈中”的顶点，说明该弧与栈上的路径形成有向环；到达已经完成的顶点本身并不表示有环。单一 `visited` 只能防止重复访问，不能替代三色状态判断。无向图的回边判断还需排除通往当前双亲的那条树边。

条件辨析·visited 不足以判有向环

到达已完成结点不说明有环；回到当前递归栈中的结点才形成回边。三色状态区分发现与完成。

6.10 关键路径：事件与活动的时间不同

AOE 网用顶点表示事件、带权有向边表示活动，边权为持续时间。某事件要等全部进入活动都完成才能发生；它发生之后，其出边活动才能开始。以下假设该网无环、持续时间非负，且有统一的源点和汇点；若源点或汇点不止一个，可添加零时长的虚边统一。

事件最早发生时间 `ve[v]` 按拓扑序计算：从所有前驱中取 `ve[u]+w(u,v)` 的**最大值**，因为全部前导活动都得完成。令汇点的最迟发生时间等于总工期，事件的最迟发生时间 `vl[u]` 则按逆拓扑序计算，取所有后继的 `vl[v]-w(u,v)` 的**最小值**，以保证每条后续活动仍赶得上。

活动 $u \to v$ 的最早开始时间为 $e = ve[u]$，最迟开始时间为 $l = vl[v] - w(u,v)$。$l - e$ 是该活动的时间余量；余量 0 的活动为关键活动。关键路径是源到汇的一条最长带权路径，决定最短可完成工期；它不是边最多的路径。

6.10.1 例 6-6　多条关键路径时怎样缩短工期

例 6.10.1 (配套案例)

【配套例题】下图 AOE 网包含 8 个活动。通过同时加快若干活动，可缩短整个工程工期。下列选项中，加快所列活动进度就可缩短工期的是（　）。

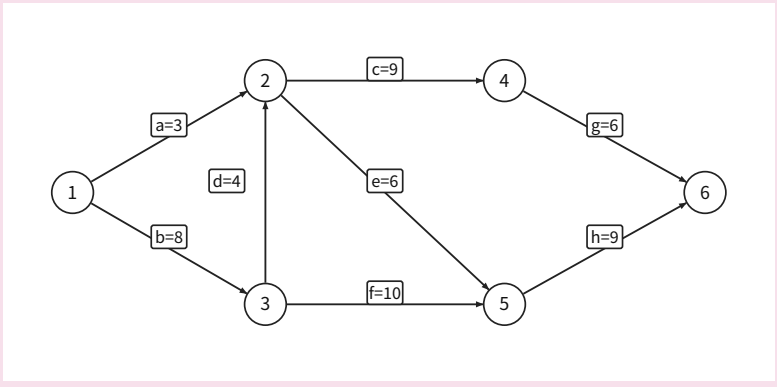


Figure 6.6: 箭头旁给出活动名和持续时间；d 的方向为 3→2

A. c 和 e　　B. d 和 c　　C. f 和 d　　D. f 和 h

解析。先按拓扑次序 1、3、2、4、5、6 算事件时间。

事件	ve	vl
1	0	0
2	12	12
3	8	8
4	21	21
5	18	18
6	27	27

例如 `ve[2]=max(0+3,8+4)=12`，不能仅沿 1→2 算成 3；`ve[5]=max(12+6,8+10)=18`。从汇点反推，`vl[2]=min(21−9,18−6)=12`，同时约束它通往 4 和 5 的两条后续活动。

活动 a 余量为 9，其余 b、c、d、e、f、g、h 均为关键活动。关键路径有三条：`b-d-c-g`、`b-d-e-h`、`b-f-h`，长度均 27。若只缩短其中一条路径上的活动，其他路径仍可能保持 27。f 与 d 合起来覆盖三条关键路径：d 影响前两条，f 影响第三条；两者同时适当缩短，所有原关键路径都会变短。答案为 **C**。A、B 遗漏路径 b-f-h，D 遗漏路径 b-d-c-g。

增加关键活动时长会使包含它的路径变长，进而延长工期；缩短某一关键活动却未必缩短工期，因为还可能有不包含它的其他关键路径。持续缩短还可能使原非关键路径成为新的关键路径，必须重新计算，不能无限按原余量推断。

错项排除·缩短一条关键活动未必够

存在两条等长关键路径时，只缩短其中一条独有活动，另一条仍保持原工期。要检查所有当前关键路径是否都被影响，并在缩短后重新计算新关键路径。

6.10.2 关键路径的计算表怎样落到程序

```
criticalPath(G):
    取得拓扑序T；若不存在则模型非法
    ve[*]=0
    按T正序，对每条u到v：ve[v]=max(ve[v],ve[u]+w(u,v))
    D=ve[统一汇点]；vl[*]=D
    按T逆序，对每条u到v：vl[u]=min(vl[u],vl[v]-w(u,v))
    对每条活动u到v:
        e=ve[u]；l=vl[v]-w(u,v)
        若e==l：该边为关键活动
```

按“全部前驱都完成”取最大最早时间，按“不拖延任何后继”取最小最迟时间，方向不能交换。不能将 `ve[v]`直接当作边 $u \to v$ 的活动最早开始时间，活动开始由弧尾事件 u 决定。

任务	前提与优化对象	核心状态	典型边界
BFS 最短路	同权边；最少边数	分层队列和距离	非等权不能直接用
MST	带权无向图；总树权最小	Prim 跨割值或 Kruskal 分量	不保证源点路径最短
Dijkstra	非负权；单源路径和最小	已确定集合与暂定上界	负边破坏通用保证
Floyd	无相关负环；点对最短	允许的中转点集合	中转点循环在最外层
拓扑	DAG；满足先后约束	余图入度	无候选而未结束表示有环
关键路径	AOE DAG；总工期	事件 ve/vl 及活动 e/l	多条关键路径需同时考虑

6.11 主动回忆自测

以下为自拟补充，检查算法选择的边界。

1. 图有 8 个顶点、7 条边，能否直接认定为树？若再给“连通”呢？

2. 有向图 A→B、A→C、B→D、C→D 有几个拓扑序列？若添加 B→C，怎样变化？
3. 一条边权变大，MST 总权与某一对顶点最短距离是否一定同时改变？
4. 一条关键活动缩短 1，工期是否必缩短 1？
5. 无向图三点 A、B、C，边 AB=1、AC=3、BC=1。从 A 作 BFS 得到的到 C 路径是否必为最短权路径？
6. Prim 的 `key[v]` 与 Dijkstra 的 `dist[v]` 分别保存什么？为什么松弛公式不同？
7. Floyd 把 `k` 放内层，仅执行一次三重循环还能直接沿用原证明吗？
8. DAG 中某轮有两个零入度顶点，能否仍有唯一拓扑序列？

答案。1. 不能；若无向且连通，7 条边恰够生成树，不可能再有环，故为树。2. 原有 A-B-C-D 与 A-C-B-D 两个；添加后仅 A-B-C-D。3. 不一定；该边可能不在任何当前采用的最优解中，两种优化目标也不同。4. 不一定；有另一条不含该活动的同长关键路径时工期不变。

5. 不能；BFS 可一条边直接到 C，权为 3，实际 A-B-C 权为 2。它最少的是边数，不是总权。
6. 前者是连接现有树集合到 v 的最轻边；后者是从固定源点到 v 的已知路径最小上界，后者需累加已确定前缀距离。
7. 不能，阶段已不再对应统一的中转集合；必须重新证明算法，而非只因循环次数相同就认定等价。
8. 不能；在无环余图中这两个候选可分别作为下一项，并可扩展为合法序列，至少产生两种拓扑序。

错项排除·缩短一条关键活动未必够

存在两条等长关键路径时，只缩短其中一条独有活动，另一条仍保持原工期。要检查所有当前关键路径是否都被影响，并在缩短后重新计算新关键路径。